

Code-Layout, Readability and Reusability

Date	14 July 2026
Team ID	SWTID-2026-7049
Project Name	Emotion Detection & Learning Support Engine (AI Learning Assistant)
Maximum Marks	5 Marks

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP8-style, 4-space indentation across src/ and scripts/.
2	Proper File Structure	Files and folders are logically organized	Yes	src/ split by pipeline stage: inference, generation, orchestration, persistence, preprocessing, models, ui.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. bilstm_result, bert_result, trust_level, reason — read clearly at every call site.
4	Function / Method Names	Functions are descriptively named	Yes	e.g. decide_emotion(), run_analysis(), record_interaction(), _has_real_weights().
5	Code Comments	Inline and block comments explain logic	Yes	Module docstrings consistently explain why a decision was made, not just what the code does (e.g. decision_engine.py documents the exact validation numbers behind every threshold).
6	Modular Design	Code is split into reusable functions/modules	Yes	decision_engine.py is a pure function with zero UI/file/network dependency; UI is split one file per concern (9 files under src/ui).
7	No Redundant Code	Duplicate or unused code is removed	Partial	csv_prediction_toggle.py is intentionally-disabled scaffolding for an unimplemented feature — kept because it's clearly labeled "planned feature" rather than silently removed.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Graceful-degradation paths throughout: weight-presence checks before model load, a BERT-unavailable branch in the decision engine, a template fallback if the AI call fails, and a try/except around st.secrets access.

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability
1	decision_engine.py	Python (pure function)	Would be reused as-is if either model is retrained or replaced — no UI/IO coupling	High
2	schema.py (unified prediction schema)	Python	Shared by both BiLSTM and BERT predictors so downstream code never special-cases either model	High
3	cached_loaders.py	Python (@st.cache_resource)	Single load-once pattern reused for both model loads	Medium
4	llm_client.py provider abstraction	Python	Swaps between Gemini and OpenRouter without touching prompt_builder.py or response_generator.py	High
5	UI toggle modules (ai_toggle.py, save_data_toggle.py, show_details_toggle.py, csv_prediction_toggle.py)	Python / Streamlit	Each follows the same one-function-one-checkbox pattern, so adding a new setting is a copy-paste-adjust job	Medium

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear separation across 7 src/ subpackages, each mirroring an actual pipeline stage.
Readability	4	Docstrings consistently explain engineering rationale, not just mechanics; a few UI toggle files are intentionally terse (they're one-liners).
Reusability	4	Decision engine, schema, and cached loaders are cleanly reusable; prompt_builder and template_fallback are tightly coupled to this project's specific emotion/field framing.
Documentation / Comments	5	Module-level docstrings routinely record why a decision was made (e.g. why subprocess isolation, why the config.json check), which is unusually thorough for a project this size.
Overall Score	4.5	Strong engineering discipline overall; the one visible gap is the disabled CSV-prediction toggle left in place — a deliberate, documented tradeoff, not an oversight.